AWS Certified Data Engineer Associate Study Guide
In-Depth Guidance and Practice Decision Making
Sakti Mishra, Dylan Qu & Anusha Challa
O'Reilly Press

Summary
by
Sadaf Iqbal, Muhammad Suleman

Preface

Purpose & Author Background

- Written by **AWS Data Analytics Specialist Architects** with 5+ years solving **large-scale, real-world data problems**
- Experience spans **data strategy, petabyte-scale lakehouses, cost/performance optimization, security, and governance**
- Rising **generative AI** makes strong data engineering skills a **business differentiator**

Why This Book

- Recommends **AWS Certified Data Engineer – Associate (DEA-C01)** as a **structured entry point** into data engineering
- Focus is **learning fundamentals + AWS thinking**, not just earning a credential
- Created as the **practical guide the authors wished they had**

What This Book Is *Not*

- ❌ Not a deep dive into individual AWS services
- ❌ Not a step-by-step implementation manual
- ✅ Focuses on **core concepts and architectural patterns** for AWS data engineering

What This Book Covers

- DEA-C01 **exam format, preparation strategy, and mindset**
- **Role and responsibilities** of an AWS Data Engineer
- How **AWS data, analytics, and supporting services** work together
- **Service selection trade-offs**: cost, performance, security, availability

Target Audience

- **Software engineers / data scientists / analysts** transitioning to data engineering
- **Practicing data engineers** seeking broader AWS ecosystem understanding
- Anyone preparing for **DEA-C01 certification**

Book Structure

Part 1 (Ch. 1–3): Foundations

- Data engineer role
- DEA-C01 exam overview & study plan
- Core concepts: databases, data lakes, Spark, Flink, AWS fundamentals

Part 2 (Ch. 4–7): Core Exam Domains

- Data ingestion & transformation pipelines
- Data store selection and management
- Pipeline operations & optimization
- Data security, governance, and compliance

Part 3 (Ch. 8–9): Practice & Readiness

- Hands-on batch and streaming pipeline implementation
- Full-length practice exam with explanations

Part 4 (Ch. 10): Future Outlook

- New and emerging AWS data services
- Forward-looking skills beyond current exam scope

Chapter 1: Certification Essentials — Overview

- Sets foundation for **AWS Data Engineer role** and **DEA-C01 exam**
- Focuses on **skills, exam structure, and preparation strategy**

Role of a Data Engineer

- Designs **scalable, low-latency, reliable data systems**
- Builds **ETL pipelines, data lakes, warehouses, and streaming systems**
- Bridges **raw data → analytics, ML, BI, and applications**
- Ensures **data quality, reliability, and performance at petabyte scale**

AWS in Data Engineering

- Provides end-to-end data capabilities: **processing, analytics, streaming, BI**
- Core services: **EMR, Glue, Redshift, Kinesis**
- Emphasis on **price–performance, scalability, and governance**
- Focus on **orchestrating services into complete data platforms**

Why Become AWS Certified Data Engineer Associate

- **Structured learning path** for AWS data engineering
- **High market demand** and strong career growth
- Validates **best practices**: security, governance, operations, compliance
- Access to a **global AWS data community**
- **AI-era relevance**: AI quality depends on well-engineered data

Exam Scope & Domains

- Validates skills in **ingestion, transformation, orchestration, modeling, lifecycle, and quality**

Domain Weighting

- **Data Ingestion & Transformation** – 34%
- **Data Store Management** – 26%
- **Data Operations & Support** – 22%
- **Data Security & Governance** – 18%

Exam Format

- Question types:
    - **Multiple choice** (1 correct)
    - **Multiple response** (multiple correct)
- Focus on constraints like **lowest cost** or **least operational effort**
- No penalty for guessing

Exam Details

- Level: **Associate**
- Questions: **65** (50 scored, 15 unscored)
- Duration: **130 minutes**
- Cost: **$150 USD**
- Delivery: **Pearson VUE / Online proctored**

- Languages: EN, JA, KO, ZH (Simplified)

Registration & Preparation

- Register via **aws.training** → Certification → Schedule exam
- Use **AWS Skill Builder official practice questions**
- Each chapter includes **exam-style questions**
- Final chapter provides a **full practice exam**

Think Like an AWS Solutions Architect

- **AWS exams test problem-solving, not memorization**
- **Questions mirror real customer scenarios**
- **Goal: identify intent, constraints, and best-fit solution**

Solutions Architect Problem-Solving Framework

1. **Understand the use case**
    - **What problem is being solved? (streaming, migration, security, latency)**
2. **Assess current solutions & pain points**
    - **Legacy limits: cost, performance, scalability, ops effort**
3. **Define success criteria**
    - **Rank priorities (cost, latency, availability, simplicity)**
4. **Consider customer context**
    - **Industry, team skills, architecture, managed vs self-managed preference**
5. **Explore solution options**
    - **Evaluate multiple AWS services or patterns**
6. **Evaluate trade-offs & choose**
    - **Balance cost vs performance vs operational overhead**

Real-World Example: Fraud Detection (Streaming)

- **Use case: Near–real-time fraud detection**
- **Pain points: Batch delays, poor scalability, heavy ops**

- **Success criteria: Subsecond latency, scalability, low ops**
- **Context: Fintech startup, serverless preference, small team**
- **Options: Kinesis vs MSK; Spark vs Flink**
- **Final choice: Kinesis + Spark Structured Streaming on AWS Glue**
    - **Managed, scalable, low operational overhead**

Applying This to Certification Questions

- **Exam questions = compressed customer case studies**
- **Key skill: spot the dominant constraint (e.g., “least cost”, “least ops”)**

Sample Exam Question Breakdown

- **Use case: Real-time clickstream analytics**
- **Key constraint: Least operational overhead**
- **Context: Managed services + Spark familiarity**
- **Best option:**
  Amazon Kinesis Data Streams + Spark Structured Streaming on AWS Glue

Key Exam Insight

- **Eliminate answers that:**
    - **Increase operational burden**
    - **Don’t match team skills**
- **Prefer managed AWS-native services when stated**

Exam Expectation Note

- **DEA-C01 also tests hands-on knowledge**
- **Expect questions on:**
    - **AWS Glue, Redshift, Lake Formation**
    - **Configurations, tuning, troubleshooting**
- **Not just architecture—real usage experience matters**

Study Plan for DEA-C01

1. **Understand exam-style questions**
    - **Learn how AWS frames scenario-based questions**
2. **Study core concepts**
    - **Read this guide end-to-end**
    - **Complete end-of-chapter questions and Chapter 9 practice exam**
3. **Get hands-on practice**
    - **AWS Builder Labs: Console-based, guided labs**
    - **AWS Cloud Quest (Data Analytics): Role-based, gamified learning**
    - **AWS Jam: Real-world, open-ended problem solving**
4. **Validate readiness**
    - **Take the AWS Official Practice Exam after registration**

Chapter 2: Prerequisite Knowledge — Overview

- **Covers core data engineering foundations before AWS services**
- **Topics: databases, processing systems, big data concepts**

Databases: Core Concepts

What Is a Database

- **Persistent, electronic data storage**
- **Optimized for efficient querying vs flat files**

Database Management System (DBMS)

- **Software to create, read, update, delete data**
- **Handles indexes, caching, performance, admin tasks**
- **Examples: MySQL, PostgreSQL, Oracle, MSSQL, Amazon Aurora**

Types of Databases

Hierarchical Databases

- **Tree structure (parent → child)**
- **One-to-many relationships only**
- **Poor scalability, high complexity → largely obsolete**

Relational Databases

- **Tabular model: tables, rows, columns**
- **Most widely used today**

- **Query language: SQL**

SQL Categories

- **DDL: CREATE, ALTER, DROP**
- **DML: INSERT, UPDATE, DELETE**
- **DQL: SELECT**
- **DCL: GRANT, REVOKE**
- **TCL: COMMIT, ROLLBACK**

NoSQL Databases

- **Designed for variable schema and specific access patterns**
- **No joins or referential integrity**
- **Avoids sparse tables with NULLs**
- **Types: Key-value, Document, Graph, In-memory, Search**

OLTP vs OLAP

OLTP

- **Record-level operations**
- **High concurrency, low latency**
- **Frontend transactions (orders, payments)**

OLAP

- **Column-level analytics**
- **Large scans, lower concurrency**

- **BI, reporting, trend analysis**
- **Common storage: data lakes, data warehouses**

Big Data Overview

- **TB–PB scale data beyond traditional databases**
- **Requires distributed processing frameworks**
- **Enables analytics, ML, and generative AI**

The Five Vs of Big Data

- **Volume: Data size (TB–PB)**
- **Velocity: Data arrival speed (batch vs streaming)**
- **Variety: Structured, semi-structured, unstructured**
- **Veracity: Data quality and reliability**
- **Value: Business insights and impact**

Data Types

- **Structured: RDBMS, CSV**
- **Semi-structured: JSON, XML, email**
- **Unstructured: Audio, video, PDFs**

Key Challenge

- **Single-node systems don’t scale**
- **Requires distributed, parallel processing frameworks**
- **Leads to technologies like Hadoop, Spark, Flink (covered next)**

Distributed Processing Frameworks — Overview

- **Distributed processing = multi-node clusters, not single machines**
- **Enables parallel, fault-tolerant processing at big data scale**
- **Hadoop ecosystem pioneered this model**

Hadoop Ecosystem

- **Master + Data nodes, scales horizontally**
- **Storage layer: HDFS (Hadoop Distributed File System)**
- **Foundation for multiple processing frameworks**

MapReduce

- **Batch-oriented distributed processing**
- **Data split into 64–128 MB chunks**
- **Core stages:**
    - **Map: transform input key–value pairs**
    - **Reduce: aggregate intermediate results**
    - **Combiner (optional): local pre-aggregation**
- **Reliable but slower and disk-heavy**

Apache Spark

- **In-memory, optimized successor to MapReduce**
- **Supports batch, streaming, ML, graph**

- **APIs: Java, Scala, Python, R**

Spark Concepts

- **Data abstractions: RDDs, DataFrames, Datasets**
- **Transformations: lazy (map, filter, join)**
- **Actions: trigger execution and output**

Spark Architecture

- **Driver + Cluster Manager + Workers**
- **Workers run executors with fixed CPU & memory**
- **Supports in-memory caching for performance**

Apache Flink

- **Designed for real-time, stateful stream processing**
- **Handles bounded (batch) and unbounded (streaming) data**
- **Key features:**
    - **Exactly-once semantics**
    - **Out-of-order event handling**
    - **Backpressure control**

Flink Architecture

- **JobManager: scheduling, checkpoints, fault recovery**
- **TaskManager: executes tasks using task slots**
- **High availability via multiple JobManagers**

SQL on Big Data

Apache Hive

- **SQL-based analytics on big data**
- **Acts as data warehouse layer in Hadoop**
- **Uses HiveQL**
- **Stores schema in Hive Metastore (HMS)**
- **Backends: MapReduce, Tez, or Spark**

Presto

- **Distributed SQL engine for low-latency queries**
- **In-memory execution, highly parallel**
- **Queries data from HDFS, object stores, RDBMS, NoSQL**
- **Faster alternative to Hive**

Trino

- **Fork of Presto (formerly PrestoSQL)**
- **Community-driven, performance-focused**
- **Modern SQL analytics engine**

Industry Usage Summary

- **Batch: Spark**
- **Streaming: Spark Structured Streaming, Flink**
- **SQL Analytics: Hive, Presto, Trino**

Data Lake

- **Centralized storage for structured, semi-structured, unstructured data at any scale**
- **Stores raw data as-is; processing happens later**
- **Popularized by HDFS, now mainly cloud object stores (e.g., S3)**
- **Used for analytics, ML, historical analysis**
- **Risk: data swamp if metadata, structure, governance are missing**
- **Uses schema-on-read**

Data Warehouse

- **Centralized system for analytics & BI**
- **Optimized for structured / semi-structured relational data**
- **Supports low-latency SQL queries**
- **Uses proprietary, optimized storage formats**
- **Tiered storage (in-memory, SSD) and smart data distribution**
- **Uses schema-on-write**
- **May include data marts for specific teams**

Data Lake vs Data Warehouse

- **Data Lake: all data, raw + historical, ML-friendly, flexible**
- **Data Warehouse: curated data, recent years, fast analytics**
- **Most organizations need both**

- **Data lakes → programming frameworks (Spark, Flink, Hive)**
- **Warehouses → SQL-first interfaces**

ETL vs ELT

- **ETL: Transform → then Load**
    - **Best for semi/unstructured data or non-queryable formats**
- **ELT: Load → then Transform**
    - **Best for structured data, analyst-driven SQL use**
- **Choice depends on data type, latency, tools, team skills**
- **No universal winner**

Data Processing Methods

Batch Processing

- **Processes data at intervals**
- **High data volume, longer execution**
- **Best for bounded datasets**

Real-Time Stream Processing

- **Continuous processing of unbounded data**
- **Handles late/out-of-order events, checkpoints**
- **Used for clicks, IoT, CDC, SaaS updates**

Event-Driven Processing

- **Triggered when data arrives**
- **Cost-efficient when data is irregular**
- **Scales dynamically**

High-Level Data Pipeline Architecture

- **Data Sources → systems producing data**
- **Ingestion → batch, streaming, queues**
- **Distributed Storage → lake, warehouse, database**
- **Processing → validation, enrichment, transformation**
- **Catalog → metadata & data discovery**
- **Consumption → analytics, BI, ML, downstream systems**
- **Security & Governance → access, lineage, masking**
- **Orchestration → scheduling, retries, monitoring**
- **Monitoring → logs, alerts, infrastructure health**

Code Repositories

- **Centralized storage for application source code**
- **Enable collaboration, version control, security, and controlled releases**
- **Core to modern software and data engineering workflows**

Key Benefits

- **Collaboration: Multiple developers work on the same codebase**

- **Version Control: Track changes, rollback to previous versions**
- **Security: Fine-grained access (read/write/merge)**
- **Productivity: Faster development, easier maintenance**
- **Code Review & Release: Safer merges and smoother deployments**

Working with Git Repositories (GitHub Example)

- **Clone repo: git clone <repo-url>**
- **Fetch updates: git fetch <remote>**
- **Merge branch: git merge <remote>/<branch>**
- **Rebase commits: git rebase <remote>/<branch>**
- **Pull (fetch + merge): git pull <remote> <branch>**
- **Push changes: git push <remote> <branch>**

Merge Conflict Handling

- **Happens when same file + same lines are modified**
- **Steps:**
    - **Check conflicts: git status**
    - **Manually resolve conflict markers (<<<<<<<, =======, >>>>>>>)**
    - **Stage changes: git add**
    - **Commit fix: git commit -m "Resolved merge conflict"**

CI/CD Overview

- **Automates build, test, and deployment**
- **Reduces manual errors and release time**

Continuous Integration (CI)

- **Merges developer code**
- **Runs automated tests and builds**

Continuous Delivery

- **Deploys build to preproduction**
- **Validates in production-like environment**

Continuous Deployment

- **Automatically deploys to production**

Common CI/CD Tools

- **Jenkins**
- **GitHub Actions**
- **AWS CodePipeline**

Why Cloud Computing

- **Traditional on-prem servers → slow procurement, high upfront cost, poor scalability**
- **Cloud removes capacity planning and data center management**

Cloud Computing

- **On-demand access to compute, storage, networking, managed services**
- **Pay-as-you-go or reserved pricing**
- **Offered as IaaS, PaaS, SaaS**

Key Benefits

- **Agility: Rapid experimentation and deployment**
- **Elasticity: Scale up/down automatically, serverless support**
- **Cost efficiency: Pay only for usage**
- **Global reach: Deploy in minutes across regions**

Amazon Web Services (AWS)

- **Cloud platform with 200+ services**
- **Purpose-built managed services reduce ops overhead**
- **Cost optimization via Savings Plans, Reserved Instances**
- **Global infrastructure enables high availability**

AWS Global Infrastructure

- **Regions: Geographically isolated locations**
- **Availability Zones (AZs): Multiple isolated data centers per region**
- **Services can be:**
    - **Regional (most services)**

    - **Global (e.g., IAM)**

- **Multi-AZ services → built-in resilience**
- **Data transfer costs apply across AZs and Regions**

Getting Started with AWS

1. **Create AWS account**
2. **Configure IAM access**
3. **Choose primary Region**
4. **Provision compute & storage**

Operational Best Practices

- **Cost tracking (Budgets, Cost Explorer)**
- **Resource tagging**
- **Logging (CloudWatch), auditing (CloudTrail)**
- **Disaster recovery (RPO, RTO)**
- **Security best practices**
- **Cost optimization (autoscaling, serverless)**

AWS Account Setup Best Practices

- **Use corporate email & phone**
- **Enable MFA for root user**
- **Use AWS Organizations & Control Tower for multi-account**

- **Understand Shared Responsibility Model**
- **Leverage AWS Free Tier**

AWS IAM (Identity and Access Management)

- **Global service for authentication & authorization**
- **Manages users, groups, roles, policies**
- **Permissions defined via JSON policies**

IAM Users

- **Created via console, CLI, or API**
- **Follow least privilege**
- **Enable MFA**
- **Avoid long-term credentials where possible**
- **Can integrate with Active Directory / Identity Center**

IAM Policies

- **Group permissions into reusable units**

Types

- **AWS-managed: Prebuilt, not editable**
- **Customer-managed: Custom, reusable**

- **Inline: Attached directly to one user/role**

IAM Roles

- **Assume-based access with temporary credentials**
- **Attach multiple policies**
- **Used by users, services, or external identities**

Benefits

- **Reduced credential risk**
- **Easier permission management**
- **Secure delegation of access**

IAM Best Practices

- **Enable MFA**
- **Prefer roles over direct user permissions**
- **Enforce least privilege**
- **Use federated access for humans**
- **Centralize access via IAM Identity Center**
- **Add metadata for auditing**

CHAPTER 3 Overview of AWS Analytics and Auxiliary Services

AWS Analytics Overview

- **AWS provides a broad set of managed analytics services for processing, analyzing, and visualizing data efficiently.**
- **Supports data lakes, warehouses, real-time streaming, and machine learning integration.**
- **Enables scalable, flexible, cost-effective analytics for modern data architectures.**

Real-Time Streaming Services

Amazon Kinesis Data Streams

- **Fully managed, real-time data streaming service.**
- **Collects, processes, and analyzes data from clickstreams, IoT devices, logs, and transactions.**
- **Features**
    - **Easy administration: fully managed infrastructure.**
    - **Native AWS integration: Lambda, Firehose, IoT Core, CloudWatch.**
    - **On-demand autoscaling: no manual capacity planning.**
    - **Flexible retention: 24 hours–365 days.**
    - **Enhanced fan-out: low latency per consumer.**
- **Use Cases**
    - **Streaming log/event data.**
    - **Event-driven applications (fraud detection, monitoring, personalization).**
    - **Moving from batch to real-time analytics.**

Amazon Data Firehose

- **Managed streaming ETL service for near real-time ingestion into S3, Redshift, or third-party services.**
- **Features**
    - **No-code setup, serverless, automatic scaling.**
    - **Lightweight transformations via Lambda.**
    - **Integrates with AWS services and third-party tools like Splunk, Snowflake, Datadog.**
- **Use Cases**
    - **Load streaming data into data lakes/warehouses.**
    - **Stream logs to SIEM tools for monitoring.**

Amazon Managed Service for Apache Flink

- **Fully managed Flink service for stateful, low-latency, real-time streaming applications.**
- **Features**
    - **Full Flink compatibility (batch + stream processing, stateful computing, exactly-once processing).**
    - **Studio notebook for interactive development (SQL, Python, Scala).**
    - **Multi-language APIs and AWS service connectors.**
- **Use Cases**
    - **Real-time analytics for user activity, IoT sensors.**
    - **Fraud detection and anomaly detection.**
    - **Event-driven architectures (dynamic pricing, personalized recommendations).**

Amazon Managed Streaming for Apache Kafka (MSK)

- **Managed Kafka service for high-throughput, low-latency event streaming.**
- **Features**
    - **Full Kafka compatibility.**
    - **Serverless scaling, tiered storage for extended retention.**
    - **MSK Connect: simplified integration with sources/destinations.**
    - **MSK Replicator: multi-region data replication.**
- **Use Cases**
    - **Real-time analytics and BI.**
    - **Event-driven microservices.**
    - **Change Data Capture (CDC) pipelines.**
    - **IoT data streaming solutions.**

Streaming Analytics Reference Architecture

- **Pattern: Real-time fraud detection with API Gateway → Amazon MSK → Apache Flink → outputs.**
- **Output destinations:**
    1. **Fraud notifications via Lambda + SNS**
    2. **Real-time search & reporting via OpenSearch**
    3. **Long-term log storage in S3**
- **Use Case: Event-driven apps, fraud monitoring, high-velocity log processing.**

AWS Glue

- **Serverless ETL & data integration service for discovering, preparing, and loading data.**
- **Features:**
    - **Scalable batch ETL (Apache Spark) and streaming ETL.**
    - **Centralized Data Catalog for schema, metadata, and quality control.**
    - **Tailored interfaces: Glue Studio (visual), notebooks (code-centric).**
    - **Built-in connectors for relational, NoSQL, warehouses, SaaS apps.**
- **Use Cases:**
    - **Data lakes creation & management**
    - **Data warehouse ETL (Redshift, Snowflake)**
    - **Real-time streaming data processing**
    - **ML data prep & feature engineering**
    - **Multi-source data integration**

AWS Glue DataBrew

- **Visual, no-code data prep tool with >250 transformations.**
- **Features:**
    - **Data profiling & quality checks**
    - **Drag-and-drop transformations**
    - **Reusable recipes for reproducibility**
- **Use Cases:**
    - **Ad-hoc exploration & analysis**
    - **BI reporting prep**

    - **Data cleaning for ML**

Amazon Athena

- **Serverless interactive SQL analytics on S3 and 30+ data sources.**
- **Features:**
    - **ANSI SQL support**
    - **Apache Spark integration for complex queries**
    - **Open table formats (Iceberg) with time travel**
    - **Federated queries across sources**
- **Use Cases:**
    - **Data lake exploration**
    - **BI dashboards & reporting**
    - **Log analysis & operational intelligence**

Amazon EMR

- **Managed big data platform supporting Hadoop, Spark, Presto, Flink.**
- **Features:**
    - **Multi-framework support**
    - **Batch, real-time, interactive, ML workloads**
    - **Flexible deployment: EC2, EKS, serverless, Outposts**
- **Use Cases:**

- **Large-scale batch ETL**
- **Real-time streaming analytics**
- **Big data analytics & ML (Spark MLlib)**

Amazon Redshift

- **Managed, petabyte-scale data warehouse with MPP architecture.**
- **Features:**
  - **Scalable & elastic (RA3 instances, serverless)**
  - **Deep AWS integration (Lake Formation, QuickSight, Redshift Spectrum)**
  - **Data sharing across clusters/accounts/regions**
  - **Integrated ML (Redshift ML with SageMaker)**
- **Use Cases:**
  - **BI & reporting**
  - **Big data analytics**
  - **ML-enabled insights**
  - **Monetizing data assets via secure sharing & Data Exchange**

Service	Type / Purpose	Key Features	Common Use Cases
Streaming Analytics Pattern	Real-time event processing	API Gateway → MSK → Flink → Outputs; Lambda+SNS, OpenSearch, S3	Fraud detection, event-driven apps, high-velocity logs

AWS Glue	**Serverless ETL & data integration**	**Batch & streaming ETL (Spark), Data Catalog, Glue Studio/notebooks, connectors**	**Data lakes, warehouse ETL, real-time processing, ML prep, multi-source integration**
AWS Glue DataBrew	**No-code data prep**	**Visual profiling, drag-and-drop transformations, reusable recipes**	**Ad-hoc exploration, BI prep, ML data cleaning**
Amazon Athena	**Serverless SQL analytics**	**ANSI SQL, Spark integration, open table formats (Iceberg), federated queries**	**Data lake analytics, BI dashboards, log analysis**
Amazon EMR	**Managed big data platform**	**Hadoop/Spark/Presto/Flink, batch/real-time/ML, EC2/EKS/Serverless/Outposts**	**Batch ETL, real-time streaming, big data analytics, ML**
Amazon Redshift	**Managed data warehouse**	**Scalable & elastic (RA3/serverless), deep AWS integration, data sharing, Redshift ML**	**BI & reporting, big data analytics, ML insights, data monetization**

Databases

AWS offers diverse database services for different data models and workloads, enabling focus on application development rather than infrastructure management. They serve as primary sources for analytics.

Service	Type	Key Features	Use Cases

Amazon RDS	**Relational**	**Managed, supports MySQL, PostgreSQL, Oracle, SQL Server**	**Transactional workloads, app databases**
Amazon Aurora	**Relational**	**High performance, separates compute & storage, multi-AZ replication**	**High-throughput apps (ecommerce, finance)**
Amazon DynamoDB	**Key-value**	**Single-digit ms latency, fully managed**	**High-velocity apps: web, gaming, ad tech, IoT**
Amazon DocumentDB	**NoSQL**	**MongoDB-compatible, JSON optimized**	**Document storage & queries**
Amazon Keyspaces	**NoSQL (Cassandra)**	**Scalable, highly available, distributed**	**Large-scale, fault-tolerant datasets**
Amazon MemoryDB	**In-memory**	**Redis-compatible, ultra-fast**	**Low-latency, high-throughput apps**
Amazon Neptune	**Graph**	**Managed graph DB, optimized for connected datasets**	**Graph-based apps, social networks, recommendation engines**

Storage

AWS storage services provide flexible, scalable solutions for different data types and access patterns, supporting analytics workloads.

Service	Type	Key Features	Use Cases
Amazon EBS	Block storage	Low-latency, high-performance	EC2-backed apps, Hadoop workloads
Amazon EFS	File storage	Elastic, NFS support, concurrent access	Shared file data across EC2 instances
Amazon S3	Object storage	Highly scalable, secure, durable	Data lakes, analytics, backup
S3 Glacier	Archival	Low-cost, durable, retrieval from ms to hours	Infrequent data access, archiving
AWS Backup	Managed backup	Centralized backup, cross-service	Disaster recovery, compliance

Machine Learning

AWS ML services support model development, deployment, and integration for predictive analytics, generative AI, and data-driven insights.

Service	Purpose	Key Features	Use Cases
Amazon SageMaker	End-to-end ML platform	Build/train/deploy models, integration with data lake & Redshift, monitoring	Predictive analytics, production ML

Amazon Bedrock	**Foundation models (generative AI)**	**Managed FMs, custom model import, RAG support, agents, guardrails**	**Generative AI apps, RAG, content safety**
Amazon Q	**AI assistant**	**Business & developer support, integrates with QuickSight, Glue, Redshift, Connect, Supply Chain**	**Natural language insights, code generation, visualization, planning**

Migration & Data Transfer

AWS provides services to migrate workloads and transfer data securely and efficiently from on-premises or other clouds.

Service	Purpose	Key Features	Use Cases
AWS DMS	**Database migration**	**Move relational/NoSQL DBs to AWS; one-time full load or continuous replication**	**Cloud migrations, database modernization**
AWS SCT	**Schema conversion**	**Converts source DB schema to target DB during migration**	**Cross-DB migrations (Oracle→Aurora, SQL Server→PostgreSQL)**
AWS DataSync	**File/object transfer**	**Secure, automated transfer from NFS/HDFS to AWS**	**Large-scale file migrations, hybrid storage**

AWS Data Exchange	**Data marketplace**	**Buy/sell datasets via secure subscriptions**	**Data monetization, data sourcing**
AWS Snow Family	**Edge/petabyte-scale transfer**	**Snowball (bulk), Snowcone (compact, edge)**	**Offline transfer, edge computing**
AWS Transfer Family	**Managed protocol transfer**	**SFTP, FTPS, AS2, FTP; integrates with S3/EFS**	**Secure file exchange**

Networking & Content Delivery

AWS networking services ensure isolation, low-latency access, and global content delivery.

Service	Purpose	Key Features	Use Cases
Amazon VPC	**Virtual network**	**Public/private subnets, isolation**	**Securely deploy workloads**
AWS PrivateLink	**Private service access**	**Access AWS services internally, no internet**	**Secure internal connections to S3, APIs**
Amazon Route 53	**DNS & routing**	**Latency/IP/geo-based routing, failover, health checks**	**Domain routing, high availability**

Amazon CloudFront	**CDN & edge computing**	**Edge caching, Lambda@Edge, S3 integration**	**Low-latency content delivery, real-time metrics**

Security, Identity & Compliance

AWS provides a comprehensive security layer across identity, data protection, threat mitigation, and compliance.

Service	Purpose	Key Features	Use Cases
IAM	**Identity & access**	**User authentication, fine-grained permissions, SSO integration**	**Secure access management**
KMS	**Encryption key management**	**Create/manage keys, multi-region, external key store support**	**Data encryption at rest**
Amazon Macie	**Sensitive data detection**	**ML & pattern matching for PII in S3**	**Data privacy, compliance monitoring**
Secrets Manager	**Credential management**	**Store DB/API keys securely, integrate with IAM**	**Avoid hardcoding secrets**
AWS Shield	**DDoS protection**	**Standard & Advanced for detection, mitigation, response**	**Protect apps from denial-of-service attacks**

AWS WAF	**Web application firewall**	**Protect HTTP/HTTPS traffic; integrates with CloudFront, ALB, API Gateway**	**Web app security, traffic filtering**

Management & Governance

AWS provides services to automate infrastructure, monitor resources, and ensure auditability.

Service	Purpose	Key Features	Use Cases
AWS CloudFormation	**Infrastructure as Code (IaC)**	**Automates creation & deployment of AWS resources**	**Consistent, repeatable deployments**
AWS CloudTrail	**Auditing & compliance**	**Tracks all API actions/events**	**Regulatory compliance, security auditing**
Amazon CloudWatch	**Logging & monitoring**	**Metrics, alarms, dashboards, log analysis**	**Performance monitoring, alerting**
AWS Config	**Resource configuration tracking**	**Audits config changes, relationships, alerts**	**Governance, compliance**
Amazon Managed Service for Prometheus	**Metrics monitoring**	**Auto-scaling, container integration**	**Observability for microservices/containers**

Amazon Managed Grafana	**Visualization & observability**	**Dashboards for metrics, traces, logs**	**Monitoring & analytics visualization**
AWS Systems Manager	**Infrastructure visibility & control**	**Node & application management, automation**	**Patch management, operational tasks**

Developer Tools

AWS developer tools streamline the software lifecycle, supporting CI/CD and infrastructure automation.

Service	Purpose	Key Features	Use Cases
AWS CLI	**Command-line management**	**Script automation for AWS services**	**Automate routine tasks**
AWS CloudShell	**Browser-based CLI**	**Pre-authenticated shell, no local setup**	**Quick terminal access**
AWS CDK	**IaC via programming**	**Use TypeScript, Python, Java, .NET, Go**	**Define infrastructure programmatically**
AWS Code Services	**CI/CD**	**CodeCommit, CodeBuild, CodeDeploy, CodePipeline**	**Version control, build, deploy, release orchestration**

Cloud Financial Management

AWS provides tools to monitor, optimize, and control cloud costs for data-intensive workloads.

Service	Purpose	Key Features	Use Cases
AWS Cost Explorer	Cost visualization & analysis	Prebuilt/custom reports, forecasting, trends	Identify savings, analyze spend patterns
AWS Budgets	Budgeting & alerts	Set thresholds, notifications via email/SNS	Prevent overspending, manage usage

AWS Well-Architected Tool (WA Tool)

- **Purpose: Helps design and review workloads against AWS best practices.**
- **Integration: Works with AWS Trusted Advisor and Service Catalog AppRegistry for assessments.**
- **Framework Pillars:**
    1. **Security – Protect workloads and data.**
    2. **Reliability – Ensure systems perform consistently.**
    3. **Operational Excellence – Improve and automate operations.**
    4. **Performance Efficiency – Optimize resource usage and performance.**
    5. **Cost Optimization – Reduce unnecessary costs.**
    6. **Sustainability – Design energy-efficient workloads.**
- **Well-Architected Lenses: Extend best practices to specific domains (ML, IoT, SAP) and industries (finance, healthcare, sports).**

CHAPTER 4 Data Ingestion and Transformation

Data Ingestion & Transformation Overview

- **Goal: Efficiently collect, ingest, and transform data to support analytics and decision-making.**
- **Core Responsibility for Data Engineers: Build reliable, optimized data pipelines for batch, near real-time, and streaming data.**
- **Key Considerations: Data source variety, volume, velocity, transformation needs, implementation complexity, performance, and cost.**

Ingestion Patterns

1. **Real-time streaming: High-velocity data requiring immediate processing (IoT, clickstreams, social media).**
2. **Batch: Periodic loads of large datasets (daily DB dumps, weekly reports). Focus on throughput over latency.**
3. **Near real-time: Semi-frequent updates with slight delay tolerance (inventory updates, price changes). Often uses CDC (Change Data Capture).**

Streaming Data Pipeline Components

1. **Stream Sources: Application logs, IoT sensors, business transactions, clickstreams.**
2. **Stream Ingestion: Collect data from multiple sources using:**
    - **Kinesis Agent (logs → Kinesis Data Streams)**
    - **DynamoDB Streams → Kinesis/MSK**

- **AWS IoT Core → Kinesis/MSK**
- **AWS DMS (CDC from DBs)**
- **MSK Connect (files, DB CDC → MSK)**
- **Custom producers via AWS SDK/KPL (high complexity)**

3. **Stream Storage: Central layer decoupling producers & consumers:**
   - **Amazon Kinesis Data Streams (KDS)**
   - **Amazon MSK (Kafka-compatible, open source ecosystem)**
4. **Stream Consumption & Processing: Transform/load data into analytics stores using:**
   - **Amazon Data Firehose (ADF)**
   - **Redshift / Redshift Serverless**
   - **Apache Flink, Spark Streaming**
   - **AWS Glue streaming jobs**
   - **Lambda / custom KCL apps**
5. **Destination Data Stores: Fast, scalable stores for analytics:**
   - **Data lakes: Amazon S3**
   - **Data warehouses: Amazon Redshift**
   - **Search/log analytics: Amazon OpenSearch**

Kinesis Data Streams vs. Amazon MSK

Attribute	KDS	MSK
Management	Low	Low (Serverless) – Medium (Provisioned)
Scalability	Seconds	Minutes
Throughput	Limited by KDS scaling	Highest
Open source	No	Yes
Data retention	Up to 365 days	Longer, tiered storage to S3
Latency	~70ms (enhanced fan-out)	Lowest

- **KDS: AWS-focused integrations, simpler ops.**
- **MSK: Open source flexibility, higher throughput, supports wider ecosystem.**

Sample Streaming Use Cases

1. **IoT → S3 Data Lake:**
    - **IoT Core → MSK → Data Firehose → S3 (Parquet/ORC, optional Lambda transformation).**
2. **Clickstream → Redshift:**

    - **KDS/MSK → Redshift → Materialized Views for real-time reporting.**

3. **DynamoDB → S3:**
    - **DynamoDB Streams → KDS → Data Firehose → S3.**
4. **AWS Logs → OpenSearch:**
    - **CloudWatch logs → Data Firehose → OpenSearch (or third-party like Splunk).**
    - **Supports backup of raw data, error handling into separate S3 locations.**

Zero-ETL Integrations

- **Goal: Minimize ETL complexity; enable near real-time, point-to-point data ingestion.**
- **Supported Targets: Amazon S3 (data lake), Amazon Redshift (data warehouse), Amazon OpenSearch (search/log analytics).**
- **Supported Sources:**
    - **AWS relational DBs: Aurora, RDS**
    - **AWS NoSQL DBs: DynamoDB, DocumentDB**
    - **SaaS apps: Salesforce, ServiceNow, Zendesk, SAP, etc.**
    - **Files from S3**
- **Advantages: Lowest operational complexity, cost-effective, reliable.**
- **Limitations: Not all source-target combinations; sub-second latency may require real-time streaming solutions.**

CDC (Change Data Capture) with AWS DMS

- **Purpose: Ingest data in near real time from databases where zero-ETL is not available.**
- **How it works:**
    - **Monitors inserts, updates, deletes.**
    - **Streams changes to target without impacting source DBs.**
- **Supported Sources: On-prem DBs (Oracle, SQL Server, MySQL, PostgreSQL, MongoDB), RDS/Aurora, Azure/Google Cloud DBs, S3.**
- **Supported Targets: S3, Redshift, RDS, DynamoDB, OpenSearch, KDS, Kafka, DocumentDB, Neptune.**
- **Schema Conversion: Use AWS SCT for one-time conversion; DMS handles ongoing CDC.**

Sample CDC Use Cases:

1. **Databases → S3 data lake: Near-real-time ingestion, Parquet preferred for analytics.**
2. **Databases → Redshift: One-time migration + ongoing CDC; incremental refresh with materialized views.**

Ingesting On-Premises Files

- **Service: AWS DataSync**
- **Features: Fast, secure transfer; parallelized; preserves metadata & permissions; supports recurring syncs; pay-as-you-go.**

Ingesting Third-Party Datasets

- **Service: AWS Data Exchange**

- **Purpose: Integrate external data (demographics, weather, finance, etc.) into analytics pipelines.**
- **Features: Marketplace for data providers, API/console access, multiple formats, robust governance.**

General Principle

- **Zero-ETL integrations are preferred whenever available: simplest, cost-effective, low operational overhead.**
- **Otherwise, follow structured best practices for streaming, database, and delivery pipelines.**

Streaming Data Ingestion (Kinesis, MSK, Firehose)

Kinesis Data Streams (KDS)

- **Capacity Modes:**
  - **On-demand: Automatic scaling, ideal for unpredictable workloads, pay-per-use.**
  - **Provisioned: Predictable workloads, fine-grained shard control.**
- **Sharding Best Practices:**
  - **Use evenly distributed, non-sequential partition keys.**
  - **Adjust shards based on throughput; split/merge as needed.**
  - **Maintain order within shards; implement shard-level checkpoints.**
- **Consuming Data:**
  - **Shared throughput: 2 MB/sec per shard shared among consumers.**
  - **Enhanced fan-out: Dedicated 2 MB/sec per consumer; reduces latency (~70 ms).**

Amazon MSK

- **Cluster Types:**
    - **Provisioned: Standard brokers (configurable, reliable), Express brokers (elastic, faster recovery).**
    - **Serverless: Auto-scaling I/O, max write 200 MiB/s, read 400 MiB/s, partitions up to 2400.**
- **Best Practices:**
    - **Select partition keys for balanced load; avoid skew.**
    - **Rightsize cluster and brokers.**
    - **Use MSK Connect for source/sink integration.**
    - **Use MSK Replicator for cross-region replication; monitor throughput quotas.**
    - **Monitor CPU (<60%), disk (<85%), memory (<60%), retention, and encryption.**

Amazon Data Firehose

- **Acts as a delivery layer from KDS/MSK to S3, Redshift, OpenSearch.**
- **Best Practices:**
    - **Tune buffer size/interval for throughput and latency.**
    - **Enable dynamic partitioning to handle traffic spikes.**
    - **Use Lambda for transformations (e.g., JSON → Parquet).**
    - **Batch and partition data for optimized delivery.**

AWS DMS (Database Ingestion / CDC)

Replication Instances

- **Use larger instances (C4, R4) for CPU/memory-intensive migrations.**
- **Multi-AZ deployment for availability.**
- **Load multiple tables in parallel; use parallel full load for large tables.**

Tasks

- **Drop indexes/constraints before full load; add afterward.**
- **Optimize LOB settings (Limited/Inline modes).**
- **Partition/filter large tables for parallel processing.**

Redshift Targets

- **Enable BatchApplyEnabled for CDC.**
- **Increase BatchApplyMemoryLimit and BatchSplitSize for efficiency.**
- **Use primary keys; avoid unsupported types (e.g., VARCHAR > 64 KB).**
- **Parallelize CDC: ParallelApplyThreads, ParallelLoadThreads.**
- **Set proper distribution/sort keys for target tables.**
- **Monitor Redshift WLM, queue disk spills, locks, and blocking sessions.**

Data Transformation Overview

- **Purpose: Integrate, clean, and transform diverse data into analyzable formats.**
- **Modes:**
    - **Batch: Large chunks processed periodically (hourly/daily/weekly). Tools: Spark (AWS Glue, EMR), SQL (Redshift).**
    - **Streaming: Continuous data processed in real time. Tools: Spark Structured Streaming, Apache Flink.**

Streaming Transformation

- **Spark Structured Streaming: Treats data as an unbounded table; SQL-like transformations; late data handling.**
- **Flink: Event-level processing; supports stateless (filter, routing) and stateful (aggregation, windowing, joins) transformations; good for real-time pattern detection (e.g., fraud).**
- **AWS Services: Glue Streaming ETL, Amazon EMR, Amazon MSF (Managed Flink).**

AWS Glue for Data Transformation

- **Capabilities: Serverless, supports batch and streaming ETL, Python shell jobs for lightweight transformations.**
- **Key Components:**
    - **Connectors: Prebuilt connectors for S3, RDS, Redshift, SaaS apps.**
    - **Bookmarks: Enable incremental processing, support CDC.**
    - **DPUs: Compute units (4 vCPU + 16 GB memory per DPU).**
    - **Worker Types: G.025X (low-volume streaming), G.2X (light transforms), G.4X/G.8X (heavy transforms/joins).**
- **Job Types:**
    - **Spark Jobs: Batch processing large datasets.**
    - **Streaming ETL Jobs: Process continuous streams with Spark Structured Streaming.**
    - **Python Shell Jobs: Lightweight, cost-efficient transformations.**
- **Interfaces:**

- **Glue Studio: Drag-and-drop visual ETL authoring.**
- **Glue Studio Notebooks: Interactive PySpark development (convert notebooks to jobs).**
- **Interactive Sessions: Real-time development/debugging against live data.**

Best Practices in AWS Glue

- **Use appropriate worker types based on workload intensity.**
- **Partition data for faster processing.**
- **Use efficient file formats: Parquet/ORC > CSV/JSON for analytics.**
- **Leverage Glue Data Catalog partitions for query pruning.**
- **Enable job bookmarks for incremental processing and recovery.**
- **Monitor jobs: Use Glue job metrics and Spark UI to detect bottlenecks and optimize performance.**
- **Use autoscaling: Scale jobs based on processing needs; maintain optimal file sizes (avoid too many small files or >1 GB).**
- **For cost efficiency, use Flex execution class for non-sensitive workloads.**

Amazon EMR for Data Transformation

- **Purpose: Flexible, scalable big data processing using frameworks like Spark, Hadoop, Hive, Presto, Flink, HBase.**
- **Processing Modes: Batch, streaming, and interactive analytics.**

Storage Options

- **Amazon S3: Recommended persistent storage; durable, cost-effective, persists after cluster termination.**

- **HDFS: Traditional Hadoop storage; fault-tolerant but adds operational overhead.**

Deployment Options

1. **EMR on EC2: Full control, multiple frameworks, customizable; Spot & Reserved Instances for cost optimization.**
2. **EMR Serverless: Managed, auto-scaling; pay only for job runtime; ideal for intermittent workloads.**
3. **EMR on EKS: Runs EMR on Kubernetes; combines flexibility, portability, multi-AZ resiliency; suited for hybrid/complex workloads.**

Instance Types

- **x86-based: Balanced compute/memory/storage (M5, R5, C5).**
- **Graviton (ARM-based): Up to 30% better price-performance for Spark workloads.**
- **Spot Instances: Cost-effective but interruptible; combine with on-demand for reliability.**

Best Practices

- **Use S3 over HDFS.**
- **Compress, compact, convert to Parquet/ORC.**
- **Partition/bucket data to reduce scanned volume.**
- **Choose instance type based on workload.**
- **Mix Spot and On-Demand instances.**
- **Enable EMR-managed scaling.**
- **Rightsize resources and use latest EMR versions.**
- **Use Serverless EMR for sporadic workloads.**

AWS Glue vs EMR

Feature	AWS Glue	EMR (EC2/EKS/Serverless)
Serverless	Yes	Serverless (only EMR Serverless/Fargate)
Frameworks	Spark, Python	Spark, Hive, Flink, HBase, Trino, etc.
Startup Time	~10s	2 min (preinitialized ~seconds)
Scaling	Fully managed	Managed autoscaling, EKS autoscaler/Karpenter
Interactive Analytics	Glue Studio/Notebooks	EMR Studio, Zeppelin, JupyterHub, Hue
Multi-AZ	No	Yes (EMR on EKS)
Cost Optimization	Flex execution	Spot, Reserved, Graviton, managed scaling
Management Overhead	Low	Medium (K8s knowledge for EKS)

Amazon Redshift for SQL-Based Transformation

- **Purpose: Petabyte-scale, SQL/PLSQL-based data warehouse.**

- **Compute Modes: Provisioned (RA3 nodes) or Serverless (RPUs).**
- **Storage: Columnar, high-performance; RMS + S3 for cold data; supports Iceberg, Hudi, Delta, CSV, Parquet.**

Multicluster Architectures

1. **Hub-and-Spoke: Separate cluster per workload (ETL, reporting, analytics); reduces interference.**
2. **Data Mesh: Cluster per business unit; full control of data assets; improves governance.**

Data Transformation Features

- **Materialized Views: Precompute query results; incremental and automatic refresh; improves dashboard/query performance.**
- **Stored Procedures: Encapsulate ETL logic; load staging → merge into target tables; supports PL/pgSQL constructs; enables delegated access control.**

Amazon Managed Service for Apache Flink (MSF)

- **Purpose: Fully managed, low-latency, high-throughput stream processing for stateful/stateless transformations.**
- **Sources: Amazon Kinesis Data Streams, Amazon MSK.**
- **Destinations: Amazon S3, Redshift, Kinesis, MSK.**
- **Best Practices:**
    - **Understand KPU-based cost model.**
    - **Avoid overprovisioning; adjust parallelism per KPU.**
    - **Use autoscaling based on metrics.**
    - **Optimize Flink code (async I/O, avoid skew).**
    - **Evaluate if Flink is needed—use Lambda for stateless/lightweight tasks.**

Amazon Data Firehose

- **Purpose: Streaming delivery service to S3, Redshift, OpenSearch.**
- **Transformations: Lightweight (JSON → Parquet, compression, batching, partitioning).**
- **Integration: Can invoke AWS Lambda for moderate transformations.**

AWS Lambda for Streaming

- **Purpose: Event-driven, short-duration (<15 min) transformations.**
- **Use Case: Format conversions, basic filtering, small aggregations.**
- **Pros: Serverless, scalable, cost-efficient, minimal operational overhead.**

Spark Structured Streaming

- **Services:**
    - **AWS Glue (serverless)**
    - **Amazon EMR (provisioned or serverless)**
- **Use Case: Micro-batch transformations, supports schema evolution, integrates with Spark ecosystem.**

Choosing the Right Streaming Transformation Service

Criteria	Firehose + Lambda	Spark Streaming (Glue/EMR)	Flink (MSF/EMR)
Transformation Type	Simple, stateless	Stateless/stateful, micro-batch	Rich stateless/stateful
Latency	Moderate	Low	Lowest
Throughput	Moderate	High	Optimized for high throughput
Exactly-once	No	With config	Native support
Out-of-order events	Limited	Limited	Efficient handling
Schema Evolution	Limited	Yes	No
Spark Ecosystem Integration	No	Yes	No
Ease for Spark users	Yes	Yes	Steeper learning curve
Operational Overhead	Low	Medium	Medium (MSF: low)

Choosing the Right Batch Transformation Service

Criteria	AWS Glue	Amazon EMR	AWS Lambda	Amazon Redshift
Suitable Use Cases	Spark-based ETL, batch pipelines, data format conversions, serverless simplicity	Any big data processing (Spark, Hadoop, Hive, Hudi, Presto, Flink), complex large-scale transformations, high performance	Event-driven, lightweight transformations (<15 min), small aggregations/filtering	SQL-based analytics, data warehouse workloads, sub-second latency reports, large-scale aggregations
Complexity of Transformation	Large-scale Spark jobs or lightweight Python shell scripts	Complex, large-scale transformations leveraging full capabilities of Spark/Flink	Simple, short-running, event-driven transformations	Complex SQL transformations on massive structured datasets
Customization & Control	Managed service, limited customization	High customization; can integrate custom libraries & AWS services	Limited customization beyond function code	Serverless or provisioned; provisioned gives more control
Infrastructure Management	Fully managed, no servers to provision	Some cluster management needed; EMR Serverless reduces overhead	Fully managed, serverless	Fully managed; provisioned mode requires node selection, serverless

				abstracts infrastructure
Expertise Required	**Low to moderate; suitable for general ETL engineers**	**High; expertise in big data framewo**	**Low to moderate; suitable for developers familiar with serverless functions**	**Low to moderate; suitable for data engineers with SQL skills**

Data Preparation for Nontechnical Users

AWS Glue DataBrew

- **Purpose: Enables nontechnical users (analysts, business users, data scientists) to clean, prepare, and transform data without coding.**
- **Interface: Fully managed, low-code visual interface with 250+ built-in transformations.**
- **Exam Tip: Know common functions and their roles for the certification.**

Handling Missing Values

- **Fill Missing Values: Replace with constant, mean/median/mode, previous/next non-missing value.**
- **Drop Rows: Remove rows containing missing values.**
- **Impute Values: Advanced techniques like KNN or decision tree imputation.**

Duplicate Records

- **FLAG_DUPLICATE_ROWS: Marks rows that are exact duplicates; first occurrence remains unflagged.**

Formatting Functions

- **Text & Date: Case changes (UPPER/LOWER/SENTENCE), quotes, date formatting.**
- **Extraction: Pull data via delimiters, positions, regex, nested structures.**
- **Replacement: Remove, replace text between delimiters/positions, standardize values.**

Combining Data from Multiple Sources

- **Union & Join: Merge datasets for integrated analysis.**

Nesting & Unnesting Complex Structures

- **Nesting: Consolidate multiple columns into array (NEST_TO_ARRAY), map (NEST_TO_MAP), or struct (NEST_TO_STRUCT).**
- **Unnesting: Flatten arrays, maps, or structs into rows/columns (UNNEST_ARRAY, UNNEST_MAP, UNNEST_STRUCT, UNNEST_STRUCT_N).**

Protecting Sensitive Data (PII)

- **Hashing & Encryption: CRYPTOGRAPHIC_HASH, ENCRYPT, DETERMINISTIC_ENCRYPT/DECRYPT.**
- **Masking: MASK_CUSTOM, MASK_DATE, MASK_DELIMITER, MASK_RANGE, REPLACE_WITH_RANDOM_BETWEEN/DATE.**
- **Row Shuffling: SHUFFLE_ROWS to remove correlations.**

Other Advanced Transformations

- **Outlier detection, column manipulations (merge/split/flag), numeric & phone formatting.**
- **Data science transformations: binarization, one-hot encoding.**
- **Mathematical, text, date, window-based, and web-specific (IP, URL) transformations.**

Orchestrating Data Pipelines in AWS

Orchestration: Coordinated execution of data workflows to ensure ingestion, transformation, analysis, and delivery steps run in the correct sequence.

Key AWS Orchestration Services

1. **AWS Step Functions**
    - **Serverless workflow orchestration using state machines.**
    - **Visual interface to coordinate AWS services like Lambda, Glue, and Redshift.**
    - **Use cases: ETL automation, parallel workloads, microservices coordination, ML model pipelines.**
    - **Key concept: States represent tasks; workflows = series of states.**
2. **Amazon MWAA (Managed Workflows for Apache Airflow)**
    - **Fully managed Apache Airflow for orchestrating AWS and non-AWS workflows.**
    - **Workflows = DAGs (Directed Acyclic Graphs), written in Python.**
    - **Components:**
        - **Tasks: Individual units of work, executed via operators.**
        - **Operators: Define work (PythonOperator, BashOperator, S3FileTransferOperator, RedshiftDataOperator, etc.).**
        - **Dependencies: Ensure tasks run in the correct order (upstream/downstream, cross-DAG, external sensors).**
    - **Sample Use Case:**
        - **Movie ratings and tags arrive in S3 → create Athena tables → join tables → clean CSV → load into Redshift every 10 minutes.**
        - **Tasks linked using `>>` operator to define execution order.**

Other Orchestration Options

- **AWS Glue Workflows: Orchestrate ETL jobs in Glue.**
- **Amazon Redshift Query Scheduler: Schedule SQL-based data processing.**
- **Amazon EventBridge: Trigger workflows based on events.**

AWS Orchestration Services

1. AWS Glue Workflows

- **Purpose: Orchestrate Glue crawlers and ETL jobs.**
- **Features:**
    - **Start triggers: schedule, on-demand, or EventBridge events.**
    - **Run properties: share state (name/value pairs) across jobs.**
    - **Visual UI or API-based creation.**
- **Use Case: Combine and transform datasets from multiple sources (e.g., ACH & check payments in S3).**
- **Cost: Only pay for underlying Glue components; no extra orchestration cost.**

2. Amazon Redshift Scheduler

- **Automates SQL statements at specific times/intervals.**
- **Use Cases: Refresh materialized views, recurring analytics, maintenance tasks.**
- **Pros: Simple, operationally lightweight for standalone SQL jobs.**

3. Amazon EventBridge

- **Purpose: Event-driven orchestration via a serverless event bus.**
- **Key Components:**
    - **Event bus: routes events from sources to targets.**

    - **Rules: event-driven or schedule-driven actions.**
    - **Targets: Lambda, Glue workflows, Step Functions, Redshift, SageMaker, API Gateway, Firehose, Kinesis, SNS, SQS.**
- **Use Case Example: Real-time S3 ingestion → EventBridge triggers Lambda → schema validation → processed data layer.**
- **Benefits: Decoupling, scalability, serverless, faster integration, flexible event routing.**

4. Choosing the Right Service

Consideration	Recommendation
Glue-specific ETL orchestration	Glue Workflows
SQL scheduling only	Redshift Scheduler
AWS services only, no external dependencies	Step Functions
External dependencies or open-source preference	MWAA (Airflow)
Event-driven orchestration	EventBridge

Coding & Interfaces

- **Step Functions: Amazon States Language (ASL) + visual UI**

- **MWAA: Python + visual DAG interface**
- **Glue Workflows: UI or JSON**
- **Redshift Scheduler: cron/at format, no visual UI**
- **EventBridge: cron or rate-based schedules, event rules**

Monitoring & Support

- **MWAA: best logging/debugging, strong open-source community**
- **Step Functions, Glue, Redshift, EventBridge: AWS monitoring, proprietary support**

Cost Summary

- **Glue Workflows & Redshift Scheduler: only pay for underlying services**
- **Step Functions: pay-per-use**
- **MWAA: higher up-front but cost-effective for complex workflows**

CHAPTER 8 Implementing Batch and Streaming Pipelines

AWS Batch Processing Pipeline – Hands-On Guide

Pipeline Overview

- **Definition:** Sequence of steps to refine, transform, and make data available for analytics.
- **Use Cases:**
    - Data cleansing & quality improvement
    - Aggregation & business rule transformations
    - Formatting for ML or BI
    - Creating optimized data models
    - Sharing formatted datasets with downstream systems

High-Level Architecture

1. **Data Sources:** Raw datasets uploaded to S3.
2. **Ingestion & Transformation:** Glue PySpark ETL triggered by EventBridge.
3. **Data Storage:** Redshift Serverless for structured storage.

4. **Analytics & Visualization:** QuickSight dashboards for BI insights.

Step-by-Step Implementation

1. **Create S3 Bucket**
    - Upload raw sales CSV files (e.g., “ch8-ex1-input-data”).
2. **Create Redshift Serverless Cluster**
    - Workgroup: “ch8-ex1-redshift-serverless”
    - Namespace: “salesdata”
    - Attach IAM role with S3 access.
    - Ensure inbound access for port 8182.
3. **Glue Data Connection**
    - Create JDBC connection to Redshift using cluster details.
    - Configure VPC, subnet, and security group.
4. **Glue PySpark ETL Job**

- Visual ETL in Glue Studio: S3 → Change Schema → Redshift target.
- Replace spaces in column names (e.g., opportunity stage → opportunity_stage).
- Run job; monitor status (Waiting → Running → Succeeded).

5. **Verify Data in Redshift**
   - Use Redshift Query Editor v2 to run SELECT queries on ingested table.
6. **QuickSight Visualization**
   - Create IAM role for QuickSight execution with:
     - **VPC permissions:** ec2:Create/Delete/Describe network interfaces, Describe VPCs/subnets/security groups
     - **Redshift permissions:** DescribeClusters, DescribeClusterSubnetGroups, DescribeClusterSecurityGroups
     - **Redshift Serverless permissions:** GetWorkgroup, ListWorkgroups

- Follow least privilege principle; restrict resources where possible.

AWS Batch Processing Pipeline – QuickSight Integration

QuickSight Setup

1. **Sign Up**
    - In AWS Console, search for QuickSight → SIGN UP
    - Authentication: IAM federated identities & QuickSight-managed users
    - Select the AWS region of your Redshift Serverless cluster
2. **Add VPC Connection**
    - Manage QuickSight → Manage VPC Connections → Add VPC Connection
    - Configure:
        - Name & VPC ID (same as Redshift)
        - Execution Role: quicksight-redshift-vpc-access
        - Subnet IDs for all AZs

        - Redshift security group

    - Wait for status: UNAVAILABLE → AVAILABLE

3. **Attach IAM Role**

    - Manage QuickSight → Security & permissions → Use existing role → quicksight-redshift-vpc-access

Create Dataset & Visualization

1. **Connect to Redshift**

    - QuickSight → Datasets → Create Dataset → Redshift Manual Connect

    - Enter Redshift connection details (same as Glue connection)

    - Validate connection → Create Data Source

    - Select Schema: public, Table: sales

2. **Load Data into SPICE**

    - Choose “Import to SPICE for quicker analytics”

    - Click Visualize

3. **Build Visualization**
    - Canvas → Area Line Chart
    - X-Axis: region
    - Value: forecasted_monthly_revenue
    - Color: segment
    - Publish Dashboard → Name: sales-data-forecast

Outcome

- Interactive dashboard showing forecasted monthly revenue by region and segment.
- Provides insights from Redshift data ingested and transformed via Glue ETL.

✅ Complete Batch Pipeline Stack

- **Data Storage:** S3 → Redshift Serverless
- **ETL Processing:** Glue PySpark
- **Orchestration:** EventBridge scheduler

- **Analytics & Visualization:** QuickSight dashboard
- **Security:** IAM roles & VPC connections
- Fully serverless and cost-efficient for end-to-end batch analytics.

Best Practices & Optimization Techniques

Security

- Use **AWS Secrets Manager** for Redshift credentials (avoid hardcoding).
- Restrict **security groups** to specific clusters.
- Apply **least-privilege IAM policies** targeting only necessary resources.

Performance

- **Partition S3 datasets** to improve query performance.
- Use **QuickSight SPICE** for faster dashboard visualizations.

Cost Optimization

- Enable **Glue job autoscaling** to prevent overprovisioning.
- Set **base and max RPUs** for Redshift Serverless to control expenses

Real-Time Streaming Pipeline (AWS)

Architecture Overview

- **Producer:** Kinesis Data Generator (KDG)
- **Buffer/Message Bus:** Kinesis Data Streams (KDS)
- **Consumer:** EMR Serverless with Spark Structured Streaming
- **Storage:** Amazon S3 (Iceberg format)
- **Query/Analysis:** Amazon Athena

Step-by-Step Implementation

1. **Create Kinesis Data Stream**
    - Name: ch8-kinesis-stream
    - Status: Active → receives streaming records
2. **Set up Kinesis Data Generator (KDG)**
    - Deploy Cognito user via CloudFormation
    - Create record template (e.g., Product Inventory JSON)

- Send test data → verify in KDS Monitoring tab

3. **Create S3 Buckets**
   - s3://ch8-ex2-streaming-checkpoint → for Spark checkpointing
   - s3://ch8-ex2-iceberg-data-lake → final processed data
   - s3://ch8-ex2-scripts → store PySpark script
4. **EMR Serverless Application**
   - Create EMR Studio → launch Serverless Spark application
   - Configure network: VPC, subnets, security groups
   - Copy Application ID for submitting Spark job
5. **VPC Endpoints**
   - EMR Serverless, Kinesis Data Streams, Amazon S3 endpoints for VPC connectivity
6. **Create Iceberg Table in Athena**
   - Database: icebergdb

- Table: productcatalog with schema matching Kinesis JSON
- Location: S3 bucket for Iceberg data

7. **PySpark Streaming Application**
   - Reads JSON from Kinesis → parses → writes to Iceberg on S3
   - Handles checkpointing to resume from failure
   - Uses Spark configurations for executors, memory, and cores
8. **Submit Spark Job via AWS CLI**
   - Mode: STREAMING
   - Retry policy: maxFailedAttemptsPerHour = 5
   - Monitored via EMR Serverless console → Spark History Server
9. **Validate Output**
   - S3 Iceberg Data Lake → data/ (Parquet) + metadata/ (JSON)
   - Query using Athena → tabular view of streaming data